

1장 프로그래밍의 개념

프로그램이란?

범용적 기계. “프로그램”을 사용하기 때문에 범용적.

프로그램이 없다면? -> 유용하지 않음

계산기 - 전용 기계 단순히 계산만 하는 게 X

컴퓨터 - 범용 기계 단순히 계산. 데이터를 처리하는 기계

프로그램 : 특정 작업을 위한 작업 지시서 - 명령어(instructions)로 구성되어 있음

프로그램의 역사

프로그래밍이 가능한 최초의 기계 : 해석기관 by 찰스 배비지. 수천개의 기어, 바퀴, 축, 레버가 증기로 작동

ENIAC : 스위치에 의해 기억. 프로그램을 변경할때마다 스위치를 처음부터 다시 연결

폰노이만 구조

프로그램은 **메인 메모리**에 저장된다 -> 쉽게 변경가능

메인 메모리에 저장된 프로그램에서 명령어들을 순차적으로 가져와서 실행.

최초의 프로그래머

에이다 러브레이스. 대문호 바이런의 친딸

배비지의 해석 기관에 매료되어 해석 기관을 위한 프로그램 개발.

서브루틴, 루프, 점프 등 핵심적인 프로그래밍 기본 원리를 고안

컴퓨터의 장점

빠름. 반복을 잘함

컴퓨터가 이해하는 언어

컴퓨터는 0과 1로 구성되어 있는 **기계어**(2진수)만 이해함.

기계어 <--컴파일러--> 프로그래밍언어 <--> 자연어

컴파일러 : 인간과 컴퓨터 사이의 통역.

인터프리터 : 한줄한줄 실행. 변경이 잦을 때.

프로그래밍 언어의 분류

기계어 : 특정 컴퓨터 명령어를 이진수로 표시한 것. 0과 1로 구성. 하드웨어 종속

어셈블리 언어 : 영어 약자로 CPU 명령어 표기. 기계어보다는 높은 수준. 기호와 CPU명령어 일대일 대응

어셈블리 : 기호를 이진수로 변환하는 프로그램

고급 언어 : 특정한 컴퓨터의 구조나 프로세서에 무관하게 독립적으로 프로그램을 작성할 수 있는 언어

C, C++, JAVA ...

컴파일러 : 고급 언어 문장을 기계어로 변환하는 프로그램

C언어 소개

1970년대 초 AT&T의 **데니스 리치**에 의해 개발.

UNIX 운영 체제 개발에 필요해서 만들어짐.

처음부터 전문가용 언어로 출발.

C언어 특징

- 간결하다. 효율적이다. 하드웨어를 직접 제어하는 저수준 프로그래밍 + 고수준 프로그래밍 둘다 가능하다.
- 이식성이 뛰어나다. 컴퓨터 하드웨어가 어떻게 동작하는 지를 이해할 수 있다.
- but, 초보자가 배우기 어렵다. 하드웨어를 제어하기 위하여 꼭 필요한 요소인 포인터를 잘못 사용하는 경우 있다.

알고리즘

: 문제를 풀기 위하여 컴퓨터가 수행하여야 할 단계적인 절차를 기술한 것

1부터 10까지의 합을 구하는 알고리즘

- 방법 1. 1부터 10까지의 숫자를 직접 하나씩 더한다
- 방법 2. 두 수의 합이 10이 되도록 숫자들을 그룹핑하여 그룹의 개수에 10을 곱하고 남은 숫자 5를 더한다
- 방법 3. 공식을 이용해서 계산한다.

알고리즘의 기술

- 순서도 : 프로그램에서의 논리 순서 또는 작업 순서를 그림으로 표현하는 방법
- 의사코드 : 자연어보다는 더 체계적이고 프로그래밍 언어보다는 덜 엄격한 언어.

2장 프로그램 개발 과정

프로그램 개발 과정

요구사항 분석 -> 설계 -> 구현 -> 테스트 -> 유지보수

설계

- 문제를 해결하는 알고리즘을 개발하는 단계
- 순서도와 의사 코드를 도구로 사용
- 알고리즘은 프로그래밍 언어와는 무관.
- 알고리즘은 원하는 결과를 얻기 위하여 밟아야 하는 단계에 집중적으로 초점을 맞추는 것.

소스 작성

- 알고리즘의 각 단계를 프로그래밍 언어를 이용하여 기술(작성)
- 소스 프로그램 : 알고리즘을 프로그램 언어의 문법에 맞추어 기술한 것
- 소스 프로그램은 텍스트 에디터나 통합 개발환경을 이용하여 작성.
- 소스 파일이름 :test.c

컴파일

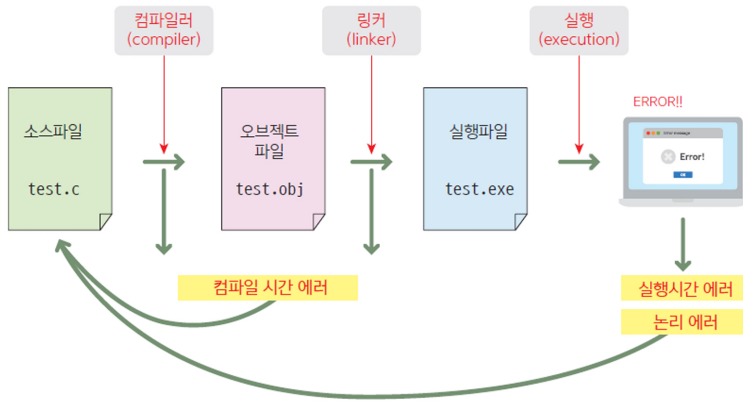
- 소스 프로그램을 **오브젝트 파일**로 변환하는 작업
- 오브젝트 파일 이름 : text.obj
- 컴파일 오류(Compile error) : 문법 오류

링크

- 컴파일된 목적 프로그램을 **라이브러리**와 연결하여 **실행 프로그램**을 작성하는 것
- 실행 파일 이름 : test.exe
- 라이브러리 : 프로그래머들이 많이 사용되는 기능을 미리 작성해 놓은 것 - 입출력, 파일처리, 수학 함수 계산 등
- 링커 : 링크를 수행하는 프로그램

실행 및 디버깅

- 실행 시간 오류(Runtime error) : 0으로 나누는 것, 잘못된 메모리 주소에 접근하는 것
- 논리 오류(logical error) : 문법은 틀리지 않았으나 논리적으로 정확하지 않은 것
- 디버깅 : 소스에 존재하는 오류를 잡는 것 - 실제 별레 때문에 명칭이 유래됨.



소프트웨어의 유지보수

이유

1. 디버깅 후에도 버그가 남아 있을 수 있기 때문
2. 소프트웨어 개발된 다음에 사용자들의 요구가 추가될 수 있기 때문

통합 개발환경

IDE = 에디터 + 컴파일러 + 디버거

VS, Eclipse, Dev-C++

솔루션 : 문제 하나에 필요한 프로젝트가 들어 있는 컨테이너

프로젝터 : 하나의 실행 파일을 만드는데 필요한 여러 가지 항목들이 등려 있는 컨테이너

헤더파일

#include : 소스코드 안에 특정 파일을 현재 위치에 포함시켜라

<stdio.h> : stdio.h 헤더파일을 포함시켜라

함수

함수 : 특정한 작업을 수행하기 위하여 작성된 독립적인 코드

오류 수정 및 디버깅

에러 : 심각한 오류

경고 : 경미한 오류

컴파일 시간 오류 : 대부분의 문법적인 오류

런타임(실행시간) 오류 : 실행 중에 0으로 나누는 연산 같은 오류

논리오류 : 논리적으로 잘못되어서 결과가 의도했던 대로 나오지 않는 오류

디버깅 : 논리 오류를 찾는 과정

F5 실행

F10 한문장씩(함수 밖으로)

F11 한문장씩(함수 안으로)

F9 중단점 설정

3장 C프로그램 구성요소

일반적인 프로그램의 형태

데이터 입력 -> 데이터 처리 -> 결과 출력

주석

```
// /* */
```

프로그램의 내용 알 수 있음.

전처리기

변수

```
int x, y, sum;           //가능
```

식별자 규칙

영문자, 숫자, 언더스코어로만 이루어짐

공백이 있으면 안 됨

첫 글자는 숫자 안됨. 영문자나 언더스코어

대소문자 구별함.

키워드와 같은 식별자는 허용되지 않음 aka 예약어

- `sum` // 영문 알파벳 문자로 시작
- `_count` // 밑줄 문자로 시작할 수 있다.
- `number_of_pictures` // 중간에 밑줄 문자를 넣을 수 있다.
- `King3` // 맨 처음이 아니라면 숫자도 넣을 수 있다.
- `2nd_base(X)` // 숫자로 시작할 수 없다.
- `money#` // #과 같은 기호는 사용할 수 없다.
- `double` // double은 C 언어의 키워드이다.

```
int width, height = 200;           //width는 초기화 안됨. height은 200으로 초기화됨
```

수식 : 피연산자와 연산자로 구성된 식

라이브러리 함수

```
printf("두 수의 합 %d\n", sum);
```

형식 지정자	의미	예	실행 결과
%d	10진 정수로 출력	<code>printf("%d\n", 10);</code>	10
%f	실수로 출력	<code>printf("%f\n", 3.14);</code>	3.14
%c	문자로 출력	<code>printf("%c\n", 'a');</code>	a
%s	문자열로 출력	<code>printf("%s\n", "Hello");</code>	Hello

```
printf("%d %f,", number, grade);
```

출력 문장	출력 결과	설명										
printf("%10d", 123);	<table><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td><td>2</td><td>3</td></tr></table>								1	2	3	폭은 10, 우측정렬
							1	2	3			
printf("%-10d", 123);	<table><tr><td>1</td><td>2</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	2	3								폭은 10, 좌측정렬
1	2	3										

출력 문장	출력 결과	설명										
printf("%f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>8</td><td></td><td></td></tr></table>	1	.	2	3	4	5	6	8			소수점 이하 6자리
1	.	2	3	4	5	6	8					
printf("%10.3f", 1.23456789);	<table><tr><td></td><td></td><td></td><td></td><td></td><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td></tr></table>						1	.	2	3	5	소수점 이하 3자리
					1	.	2	3	5			
printf("%-10.3f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	.	2	3	5						좌측 정렬
1	.	2	3	5								
printf("%.3f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	.	2	3	5						소수점 이하 자리만 표시
1	.	2	3	5								

```
scanf("%d",&x);           // &는 변수의 주소를 계산한다.
```

형식 지정자	의미	예
%d	10진 정수를 입력한다	<code>scanf("%d", &i);</code>
%f	float 형의 실수를 입력한다.	<code>scanf("%f", &f);</code>
%lf	double 형의 실수를 입력한다.	<code>scanf("%lf", &d);</code>
%c	하나의 문자를 입력한다.	<code>scanf("%c", &ch);</code>
%s	문자열을 입력한다.	<code>char s[10]; scanf("%s", s);</code>

아직 학습하지 않았음!
너무 신경쓰지 말것!

실수 입력 시 주의할 점

```
float ratio 0.0;
scanf("%f", &ratio);           // float

double scale = 0.0;
scanf("%lf", &scale);          // double
```

4장 변수와 자료형

변수와 상수

변수 : 저장된 값의 변경이 가능한 공간
상수 : 저장된 값의 변경이 불가능한 공간

char	1바이트
short	2바이트
int	4바이트
long	4바이트 //보통 int와 같음
float	4바이트
double	8바이트

unsigned : 양수만 나타냄! unsigned int

```
unsigned int speed; //부호없는 int 형
unsigned speed;     //자료형 생략하면 int형 unsigned이다!
%u                 //unsigned를 출력하기 위한 형식지정자
```

오버플로우

나타낼 수 있는 범위를 넘는 숫자를 저장하려고 할 때

상수

```
sum = 123           // 그냥 숫자만 적으면 int 형이 됨
sum = 123L          // long으로 저장됨. 대문자 L, 소문자 l 상관 X
sum = 123U          // unsigned로 저장됨. 대소문자 상관X
sum = 123UL         // unsigned long으로 저장됨.
```

진법

8진법 : 012(8)	앞에다가 0을 붙이면 8진법이다!
16진법 : 0xA(16)	앞에다가 0x를 붙이면 16진법이다!
2진법 : 0b1(1)	앞에다가 0b를

기호 상수(매크로 상수)

기호를 사용하여 상수를 표현하는 것. 가독성이 높아지고 값을 쉽게 변경할 수 있다.

두 가지 방법!

```
#define EXCHANGE_RATE 1200
const int EXCHANGE_RATE = 1200
```

정수 표현 방법

음수를 표현하기 위해서 첫 번째 비트를 부호 비트로 할 수도 있지만... 덧셈 결과가 부정확하다! >> 2의 보수법

1. 비트를 반전한다. (1의 보수)
2. 1을 더한다. (2의 보수)

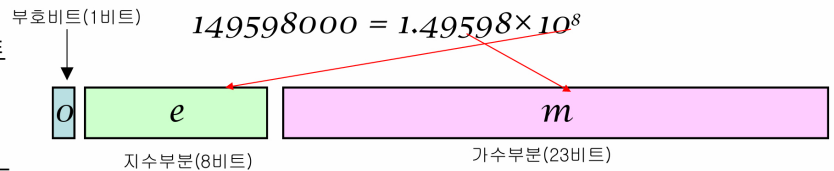
실수 표현 방법

방법 1. 정수부를 n비트 할당하고 나머지 비트를 실수부를 할당한다.

부동 소수점 방식

부호 1비트 + 지수(exp) 8비트 + 가수 23비트

$10^{-38} \sim 10^{+38}$ 까지 표현가능



float : 단일 정밀도 single precision 32비트

double : 두배 정밀도 double precision 64비트

%f : 실수를 그대로 출력

%e : exponential 형식으로 출력.

단, 표현 가능 부분까지만 출력되고 뒤는 잘린다!

- %f
 - printf("%f", 0.123456789); // 0.123457 출력
- %e
 - printf("%e", 0.123456789); // 1.234568e-001 출력

실수	지수 표기법	의미
123.45	1,2345e2	$1,2345 \times 10^2$
12345.0	1,2345e4	$1,2345 \times 10^4$
0.000023	2,3e-5	$2,3 \times 10^{-5}$
2,000,000,000	2,0e9	$2,0 \times 10^9$

```
1.23456
2.          // 소수점만 붙여도 된다.
.28         // 정수부가 없어도 된다.
2e+10       // +나 -기호를 지수부에 붙일 수 있다.
9.26E3      // 9.26×10³
0.67e-9     // 0.67×10⁻⁹
```

부동소수점 연산을 할 때는 Precision 때문에

문자형

```
char code1 ='A';
char code2 =65;    //아스키로 출력됨
```

한글은 한 글자 16비트!

제어문자(이스케이프 문자)

'\a' //알람

변수는 사용하기 전에 반드시 초기화를 해야함!

제어 문자	이름	의미
\0	널문자	
\a	경고(bell)	"삐"하는 경고음 발생
\b	백스페이스(backspace)	커서를 현재의 위치에서 한 글자 뒤로 옮긴다.
\t	수평탭(horizontal tab)	커서의 위치를 현재 라인에서 설정된 다음 탭 위치로 옮긴다.
\n	줄바꿈(newline)	커서를 다음 라인의 시작 위치로 옮긴다.
\v	수직탭(vertical tab)	설정되어 있는 다음 수직 탭 위치로 커서를 이동
\f	폼피드(form feed)	주로 프린터에서 강제적으로 다음 페이지로 넘길 때 사용된다.
\r	캐리지 리턴(carriage return)	커서를 현재 라인의 시작 위치로 옮긴다.
\"	큰따옴표	원래의 큰따옴표 자체
\'	작은따옴표	원래의 작은따옴표 자체
\\	역슬래시(back slash)	원래의 역슬래시 자체

5장 수식과 연산자

수식

상수, 변수, 연산자의 조합. 연산자와 피연산자로 나누어짐.

산술 연산자

+ - * / %

정수에서 나누기를 하면 소수점이 버려진다!

부호 연산자

x = -10

'-'는 이항 연산자이기도 하고 단항 연산자이기도 하다

증감 연산자

++ --

++x은 먼저 더하고 계산

x++은 계산하고 나중에

y = (x+1)++; // 불가능!

오류 주의

다음과 같은 수식은 오류이다. 왜 그럴까?

대입 연산자

z = x+y

상수에는 대입 불가!

++x = 10;

// 등호의 왼쪽은 항상 변수이어야 한다.

x + 1 = 20;

// 등호의 왼쪽은 항상 변수이어야 한다.

x =* y;

// *= 이 아니라 *= 이다.

복합대입연산자

x+=y

관계연산자

== != < > <= >=

- (1e32 + 0.01) > 1e32

- -> 양쪽의 값이 같은 것으로 간주되어서 거짓

★실수 비교 시 :

- (fabs(x-y)) < 0.0001

- 올바른 수식

(3 >= 2) + 5 // 참은 1으로 계산되므로 6

논리연산자

&& || !

관계 수식이나 논리 수식이 만약 참이면 1이 생성되고 거짓이면 0

!0 //1 출력

!3 //0 출력

!-3 // 0 출력

&&은 앞에게 거짓이면 뒤에 있는 피연산자는 계산하지 않는다!

||은 앞에게 참이면 뒤에 있는 피연산자는 계산하지 않는다!

조건연산자

absolute_value = (x>0) ? x : -x;

비트연산자

& 둘다 1일때 1
| 둘중 하나라도 1일때 1
^ XOR 다르면1 같으면0
~ NOT 반전!
<< 왼쪽으로 y칸만큼 이동 값 2배!
>> 오른쪽으로 y칸만큼 이동. 값 1/2배!

형변환

자동적 형변환 :

```
double f;           //올림 변환
f=10;

int i;
i = 3.141592;       //내림 변환

char c;
c=10000              //내림 변환

char와 short는 int로 변환되어 연산된다.
int + double    >> double로 변환된다.
```

명시적 형변환 :

```
(int)1.23456        //int 형으로
(double)x           //x형으로
(long) (x+y)        //long형으로
```

콤마 < 대입 < 논리 < 관계 < 산술 < 단항
괄호먼저

++ --
+ - ! ~(NOT)

곱나눗
덧뺄

비트이동
부등호
등호 (같다)

비트 AND XOR OR
논리 AND OR

삼항
대입
콤마

우선순위	연산자	설명	결합성
1	++ --	후위 증감 연산자	→ (좌에서 우)
	()	함수 호출	
	[]	배열 인덱스 연산자	
	.	구조체 멤버 접근	
	->	구조체 포인터 접근	
	(type){list}	복합 리터럴(C99 규격)	
2	++ --	전위 증감 연산자	← (우에서 좌)
	+ -	양수, 음수 부호	
	! ~	논리적인 부정, 비트 NOT	
	(type)	형변환	
	*	간접 참조 연산자	
	&	주소 추출 연산자	
	sizeof	크기 계산 연산자	
	_Alignof	정렬 요구 연산자 (C11 규격)	
3	* / %	곱셈, 나눗셈, 나머지	→ (좌에서 우)
4	+ -	덧셈, 뺄셈	
5	<< >>	비트 이동 연산자	
6	< <=	관계 연산자	
	> >=	관계 연산자	
7	== !=	관계 연산자	
8	&	비트 AND	
9	^	비트 XOR	
10		비트 OR	
11	&&	논리 AND 연산자	
12		논리 OR 연산자	← (우에서 좌)
13	?:	삼항 조건 연산자	
14	=	대입 연산자	
	+= -=	복합 대입 연산자	
	*= /= %=	복합 대입 연산자	
	<<= >>=	복합 대입 연산자	
	&= ^= =	복합 대입 연산자	
15	,	콤마 연산자	→ (좌에서 우)

6장 수식과 연산자

조건

표준적인 방법
<pre>if(x != 0) printf("x가 0이 아닙니다.\n"); if(x == 0) printf("x가 0입니다.\n");</pre>
간략한 표기법
<pre>if(x) printf("x가 0이 아닙니다.\n"); if(!x) printf("x가 0입니다.\n");</pre>

else는 가장 가까운 if절과 매칭된다!
다른 if문과 묶으려면 {}로 묶기

참고사항

실수와 실수를 비교할 때는 다음과 같은 문장을 사용하는 것은 문제가 될 수 있다.

```
if (result == expectedResult) { ... }
```

위의 비교는 참이 되기 힘들다. 왜냐하면 0.2와 같은 단순한 값은 정확하게 표현되지만 복잡한 값은 정확하게 표현되지 않기 때문이다. 따라서 부동소수점 수 2개가 같은 지를 판별하려면 다음과 같이 오차를 감안하여서 비교하여야 한다. 즉 2개의 숫자가 오차 이내로 아주 근접하면 같은 것으로 판정하는 방법이다.

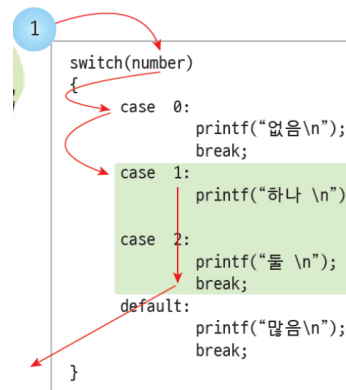
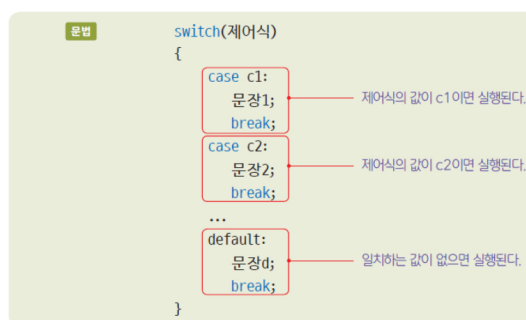
```
if (fabs(result - expectedResult) < 0.00001) { ... }
```

fabs() 함수는 실수의 절대값을 계산하여서 반환한다.

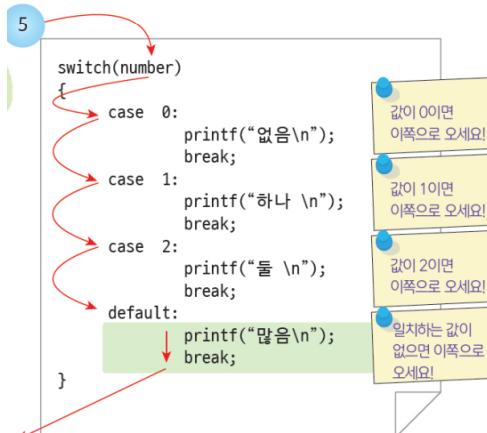
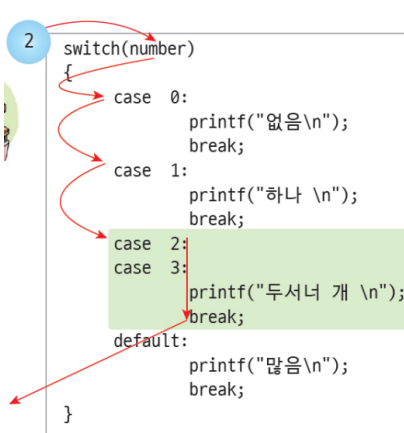
오차가 무시할 만 하면
같은 것으로 인정

Switch문

Syntax switch 문



break가 없으면 다음 break를 만날때까지 실행!



```
switch(number)
{
    case x: // 변수는 사용할 수 없다.
        printf("x와 일치합니다. \n ");
        break;
    case (x+2): // 변수가 들어간 수식은 사용할 수 없다.
        printf("수식과 일치합니다. \n ");
        break;
    case 0.001: // 실수는 사용할 수 없다.
        printf("실수 \n ");
        break;
    case 'a': // OK! 문자는 사용할 수 있다.
        printf("문자 \n ");
        break;
    case "001": // 문자열은 사용할 수 없다.
        printf("문자열 \n ");
        break;
}
```

7장 반복문

조건

```
while(i != 0)
{
    ...
}
```



```
while( i )
{
    ...
}
```

Syntax do...while 문

예



```
for ( ; ; )
    printf("Hello World!\n");
```

무한 반복 루프

```
for ( ; i<100; i++ )
    printf("Hello World!\n");
```

한 부분이 없을 수도 있다.

```
for (i = 0, k = 0; i < 100; i++ )
    printf("Hello World!\n");
```

2개 이상의 변수 초기화

```
for (printf("반복시작"), i = 0; i < 100; i++ )
    printf("Hello World!\n");
```

어떤 수식도 가능

```
for (i = 0; i < 100 && sum < 2000; i++ )
    printf("Hello World!\n");
```

어떤 복잡한 수식도 조건식이
될 수 있다.